1. equals() and equalsIgnoreCase() for String Comparison (Interview Revision)

> **Interview Goal:** This is one of the **most frequently asked Java String questions**. Interviewers often combine it with == to check whether you understand **reference comparison vs content comparison**.

---

equals()

Compares the **contents (characters)** of two Strings.

```
String s1 = "Java";
String s2 = "Java";

System.out.println(s1.equals(s2));
```

Output

```
true
```

---

equalsIgnoreCase()

Compares String contents **without considering letter case**.

```
String s1 = "JAVA";
String s2 = "java";

System.out.println(s1.equalsIgnoreCase(s2));
```

Output

```
true
```

Interview Comparison

Method	Case Sensitive
equals()	✅ Yes
equalsIgnoreCase()	❌ No

Most Asked Interview Question

== vs equals()

```
String s1 = new String("Java");
String s2 = new String("Java");

System.out.println(s1 == s2);
System.out.println(s1.equals(s2));
```

Output

```
false
true
```

Interview Reasoning

==

→ Compares **references (memory addresses)**

equals()

→ Compares **actual String contents**

> **Interviewer's expectation:** Use equals() when comparing String values. Use == only when you intentionally want to know if two references point to the **same object**.

Common Mistake

❌

```
if(username == "admin")
```

✅

```
if(username.equals("admin"))
```

Another Interview Trap

```
String s = null;

System.out.println(s.equals("Java"));
```

Output

```
NullPointerException
```

Safer approach

```
System.out.println("Java".equals(s));
```

Output

```
false
```

Interview Reasoning

Calling equals() on a known non-null literal avoids a NullPointerException.

Tricky Output Questions

Question 1

```
String s1 = "Java";
String s2 = "java";

System.out.println(s1.equals(s2));
```

Output

```
false
```

Question 2

```
String s1 = "Java";
String s2 = "java";

System.out.println(s1.equalsIgnoreCase(s2));
```

Output

```
true
```

Question 3

```
String s1 = "Java";
String s2 = new String("Java");

System.out.println(s1 == s2);
```

Output

```
false
```

Top Interview Questions

1. Difference between == and equals()?
2. Difference between equals() and equalsIgnoreCase()?
3. Which one compares references?
4. Why is "Java".equals(str) safer than str.equals("Java")?
5. When should you use == with Strings?

---

2. StringBuilder for Repeated String Modification (Interview Revision)

> **Interview Goal:** Interviewers ask this topic to check whether you understand **performance**, not just syntax.

Why StringBuilder?

Since **String is immutable**, every modification creates a **new String object**.

```
String s = "Java";
```

```
s += " Programming";
s += " Language";
```

This creates multiple temporary String objects.

Instead, use:

```
StringBuilder sb = new StringBuilder("Java");

sb.append(" Programming");
sb.append(" Language");

System.out.println(sb);
```

Output

```
Java Programming Language
```

Interviewer's Reasoning

Instead of asking:

> **Why use StringBuilder?**

They're actually checking if you know:

> **Repeated String concatenation creates many temporary objects and wastes memory. StringBuilder modifies the same object, making it much more efficient.**

String vs StringBuilder

String	StringBuilder

Immutable	Mutable
New object on modification	Same object modified
Slower for repeated concatenation	Faster
Thread-safe because immutable	Not thread-safe

StringBuilder vs StringBuffer

StringBuilder	StringBuffer
Faster	Slightly slower
Not synchronized	Synchronized (thread-safe)
Preferred in single-threaded code	Used when multiple threads modify the same object

Interview Insight

Nowadays, StringBuilder is used far more frequently because most string-building operations are single-threaded.

Common Mistake

❌

```
String result = "";

for(int i=0;i<1000;i++)
{
    result += i;
}
```

Creates hundreds or thousands of temporary String objects.

✅

```
StringBuilder sb = new StringBuilder();

for(int i=0;i<1000;i++)
{
    sb.append(i);
}
```

Much more efficient.

Tricky Interview Questions

Q1. Does append() create a new object?

No.

It modifies the existing StringBuilder.

Q2. Is StringBuilder immutable?

No.

It is mutable.

Q3. Why not always use StringBuilder?

If the text never changes, use String. It is simpler, immutable, and can take advantage of the String Pool.

Q4. When should you prefer StringBuilder?

- Loops
- Large text generation
- Frequent concatenation
- Dynamic string construction

Tricky Output Questions

Question 1

```
StringBuilder sb = new StringBuilder("Java");

sb.append(" 21");

System.out.println(sb);
```

Output

```
Java 21
```

Question 2

```
StringBuilder sb = new StringBuilder("Java");

sb.append(" Programming");

System.out.println(sb.length());
```

Output

```
16
```

("Java Programming" has 16 characters.)

Question 3

```
String s = "Java";

s.concat(" Programming");

System.out.println(s);
```

Output

```
Java
```

Top Interview Questions

1. Why is StringBuilder faster than String?
2. Difference between String, StringBuilder, and StringBuffer?
3. Is StringBuilder thread-safe?
4. Does append() create a new object?
5. When should you use StringBuilder instead of String?

⭐ Interview Insight (Very Common)

A common interview question is:

> **"Why not use StringBuilder everywhere if it's faster?"**

A strong answer is:

> "StringBuilder is optimized for frequent modifications. If a string is constant or changes rarely, String is a better choice because it's immutable, supports String Pool optimization, is inherently thread-safe due to immutability, and results in cleaner code. I choose between them based on whether the string needs frequent modification."

This shows that you understand **trade-offs**, which is exactly what interviewers want to hear rather than simply saying "StringBuilder is faster."`